GESTIONE CONTO CORRENTE

Descrizione generale del progetto

Si vuole sviluppare una web application a servizi REST per la gestione di conti correnti di una Banca. Rispetto ai servizi di business descritti di seguito realizzare:

- Lo schema concettuale e creare le relative tabelle sul DB
- Il modello concettuale delle classi Entity
- Back-end: sviluppare una web application con Spring e Spring Boot e in questo ambito:
  - Sviluppare la persistenza con JPA, Hibernate e Spring Data
  - Sviluppare servizi di business transazionali con Spring
  - Sviluppare servizi Rest con Spring Web
  - Testare i servizi REST con Postman
- Front-end: sviluppare una applicazione con Javascript e React in grado di invocare i servizi REST del back-end

Descrizione generale dei dati

Le entità in gioco sono le seguenti:

- **Utente →** Utente proprietario di conti correnti. Un utente può essere trattato dal sistema indipendentemente dagli altri dati. Di un utente bisogna trattare le seguenti informazioni:
  - idUtente (obbligatorio, univoco)
  - Nome (obbligatorio)
  - cognome (obbligatorio)
  - mail (obbligatorio, univoco)
  - telefono (facoltativo)

- **Indirizzo →** indirizzo di residenza degli utenti. Di un indirizzo bisogna trattare le seguenti informazioni:
  - idIndirizzo (obbligatorio, univoco)
  - via (obbligatorio)
  - cap (obbligatorio)
  - città (obbligatorio)
  - provincia (obbligatorio)

Descrizione generale dei dati

- **ContoCorrente →** ContoCorrente bancario. Di un conto bisogna trattare le seguenti informazioni:
  - numeroConto (obbligatorio, univoco)
  - saldo (obbligatorio)
  - dataApertura (obbligatorio)

- **Movimento →** un singolo movimento bancario su un conto corrente. Un movimento rappresenta una operazione di variazione del saldo. Di conseguenza, ogni volta che il saldo del conto cambia, va associato al conto un movimento che giustifica quella variazione. Di un movimento bisogna trattare le seguenti informazioni:
  - idMovimento (obbligatorio, univoco)
  - tipo [versamento | prelievo] (obbligatorio)
  - importo (sempre positivo, obbligatorio)
  - dataOperazione (obbligatorio)

Descrizione generale dei dati

- A quanto descritto in precedenza, va aggiunto:
  - Un conto corrente **deve** avere necessariamente un utente proprietario (intestatario del conto). Inoltre, **può** ma non deve, avere un secondo proprietario (cointestatario del conto)
  - Un utente **può** possedere **da zero** a molti conti correnti (sia come intestatario che come cointestatario)
  - Un conto corrente **può** avere **da zero** a molti movimenti. Un movimento **deve** sempre essere associato al suo conto corrente. Ogni volta che il saldo del conto viene modificato per qualsiasi ragione, va aggiunto al conto un movimento che giustifica quella variazione. Questo deve valere anche se si apre un conto con un saldo maggiore di zero.
  - Un movimento **deve** sempre essere associato ad un utente, che sarebbe l'utente che ha materialmente effettuato quella operazione bancaria. Questo perché il conto potrebbe avere 2 proprietari. Questo deve essere vero anche se il conto ha un solo proprietario. Un utente può effettuare tutti i movimenti che vuole
  - Ogni utente **deve** avere un indirizzo di residenza; un indirizzo (l'indirizzo di residenza) deve essere associato ad un solo utente. Non è possibile avere un indirizzo orfano, cioè privo di associazione verso un utente

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sull'utente

- **Registrazione nuovo utente**
  - DESCRIZIONE → registrazione di un nuovo utente. L'identificativo dell'utente deve essere generato internamente dal sistema. L'utente, per essere registrato, deve possedere tutti i dati obbligatori compreso l'indirizzo di residenza.
  - IN → l'utente da registrare
  - OUT → l'utente dopo la registrazione
  - EXCEPTION → Nel caso sia già presente a sistema un utente con lo stesso idUtente o la stessa mail sollevare un errore

- **Modifica dati utente**
  - DESCRIZIONE → modifica dei dati base di un utente. Individuare l'utente attraverso l'idUtente e sostituire tutti i suoi campi con nuovi valori. Dalla modifica è escluso l'indirizzo di residenza.
  - IN →
    - idUtente – identificativo che individua l'utente da modificare.
    - I nuovi dati dell'utente da sostituire a quelli presenti sul sistema.
  - OUT → l'utente dopo la modifica
  - EXCEPTION → nel caso la nuova mail sia già presente a sistema sollevare un errore

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sull'utente

- **Lettura dati base singolo utente**
  - DESCRIZIONE → recuperare i dati base di un singolo utente, compreso il suo indirizzo di residenza, a partire dal suo identificativo.
  - IN → idUtente - identificativo che individua l'utente da recuperare
  - OUT → i dati base dell'utente con identificativo idUtente, compreso il suo indirizzo di residenza oppure null se non è presente
  - EXCEPTION → Nessuna

- **Lettura dati salienti singolo utente**
  - DESCRIZIONE → recuperare i seguenti dati di un singolo utente: idUtente, nome, congome. Lista dei conti correnti di sua propreità (in qualità di intestatario o cointestatario). Per ogni conto: numeroConto, saldo, data ultimo movimento
  - IN → idUtente – identificativo che individua l'utente da recuperare.
  - OUT → i dati salienti dell'utente con identificativo idUtente o null se l'utente non è presente
  - EXCEPTION → nessuna

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sull'utente

- **Report utenti**
  - DESCRIZIONE → recupera la lista di tutti gli utenti registrati a sistema. Per ogni utente, ritornare: idUtente, nome, cognome, città di residenza e provincia di residenza. Saldo complessivo rispetto a tutti i conti di sua proprietà alla data corrente
  - IN → idUtente - identificativo che individua l'utente da cancellare
  - OUT → lista di tutti gli utenti o una lista vuota se non vi sono utenti registrati
  - EXCEPTION → nessuna

- **Cancellazione di un utente**
  - DESCRIZIONE → cancella un utente dal sistema insieme al suo indirizzo di residenza. E' possibile cancellare un utente dal sistema solo se non ha conti correnti associati. Se ha anche un solo conto associato (in qualità di intestatario o cointestatario) l'utente non può essere cancellato.
  - IN → idUtente - identificativo che individua l'utente da cancellare
  - OUT → nessuno
  - EXCEPTION → nel caso l'utente non sia cancellabile sollevare un errore.

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Registrazione nuovo conto**
  - DESCRIZIONE → registra un nuovo conto corrente a sistema. Il numero del conto corrente deve essere generato internamente. Il conto deve essere associato all'utente identificato con idIntestatario, il quale deve essere già registrato a sistema. Se presente il parametro idUtenteCointestatario, Il conto deve essere associato all'utente identificato con idUtenteCointestatario in qualità di cointestatario, il quale deve essere già registrato a sistema
  - Se il saldo iniziale è positivo, questo rappresenta un primo versamento effettuato all'apertura del conto. In tal caso, va aggiunto al conto il relativo movimento di versamento.
  - IN →
    - saldo – saldo iniziale del conto
    - idUtenteIntestatario – id dell'utente proprietario del conto
    - idUtenteCointestatario – id dell'utente cointestatario del conto (parametro facoltativo)
  - OUT → contoCorrente – il conto corrente appena creato. Tale conto deve avere i dati base più la lista dei movimenti. Ogni movimento della lista deve avere tutti i dati base a parte l'operatore. Al posto dell'utente l'operatore, ritornare l'idUtente dell'operatore
  - EXCEPTION → nel caso il saldo iniziale sia negativo oppure l'utente con idUtenteIntestatario non esiste, sollevare un errore. Se presente il parametro idUtenteCointestatario, e l'utente con idUtenteCointestatario non esiste, sollevare un errore.

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Modifica saldo conto**
  - DESCRIZIONE → modifica il saldo di un conto corrente. ATTENZIONE: per modifica si intende sostituzione e non incremento o decremento. Nel caso, il nuovo valore del saldo sia diverso dal precedente, bisogna modificare il saldo e aggiungere il relativo movimento alla lista dei movimenti. Se il nuovo saldo è inferiore a quello corrente, si tratterà di un movimento di prelievo, altrimenti di versamento.

    Nel caso il nuovo saldo sia uguale a quello corrente, non compiere nessuna azione e ritornare il conto corrente

    Nel caso il nuovo saldo sia negativo aggiungere un ulteriore movimento di prelievo del 5% rispetto alla variazione, che rappresenta la mora per essere andati in rosso. Esempi: se il saldo corrente è 500 euro e quello nuovo diventa -700 euro, la mora è il 5% di 700 euro. Se il saldo corrente è -700 euro e quello nuovo diventa -1000 euro, la mora è il 5% di 300 euro. In ogni caso, il conto non può scendere sotto i -5000 euro
  - IN →
    - numeroConto – il numero del conto su cui si vuole modificare il saldo
    - newSaldo – il nuovo saldo da sostituire a quello corrente
    - idUtenteOperatore – id dell'utente che ha effettuato l'operazione
  - OUT → contoCorrente – il conto corrente modificato. Tale conto deve avere i dati base più la lista dei movimenti. Ogni movimento della lista deve avere tutti i dati base a parte l'operatore. Al posto dell'utente l'operatore, ritornare l'idUtente dell'operatore
  - EXCEPTION → se l'utente idUtenteOperatore non è presente a sistema oppure non è uno dei proprietari del conto (intestatario o cointestatario) sollevare un errore. Se il saldo finale è minore di -5000 euro sollevare un errore.

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Cancellare un conto corrente**
  - DESCRIZIONE → cancella un conto corrente dal sistema. Un conto è cancellabile solo se il saldo è zero. In questo caso, cancellare anche tutti i movimenti associati. Inoltre bisogna anche eliminare l'associazione con gli utenti proprietari (non cancellare gli utenti ma eliminare l'associazione che lega gli utenti al conto)
  - IN → numeroConto – il numero del conto da cancellare
  - OUT → nessuno
  - EXCEPTION → se il conto con numeroConto non esiste sollevare un errore. Se il saldo del conto è diverso da zero, sollevare un errore.

- **Leggere i dati di un conto corrente**
  - DESCRIZIONE → recuperare i dati di un conto corrente. I dati sono: numeroConto, saldo, dataApertura, la lista di tutti gli idMovimento di tutti i movimenti , la lista degli idUtente degli utenti proprietari
  - IN → numeroConto – il numero del conto da recuperare
  - OUT → il conto corrente con numero numeroConto oppure null se non è presente
  - EXCEPTION → nessuna

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Leggere i dati salienti di un conto corrente**
  - DESCRIZIONE → recuperare i dati salienti di un conto corrente. I dati sono: numeroConto, saldo, la lista di tutti i movimenti e di tutti i proprietari. Ogni movimento deve avere tutti i dati base a parte l'operatore. Al posto dell'utente l'operatore, ritornare il nome e cognome dell'operatore. Per ogni utente proprietario tornare solo nome e cognome
  - IN → numeroConto – il numero del conto da recuperare
  - OUT → i dati salienti del conto corrente con numero numeroConto oppure null se non è presente
  - EXCEPTION → nessuna

- **Report conti**
  - DESCRIZIONE → recupera la lista di tutti i conti correnti registrati a sistema. Per ogni conto, ritornare: numeroConto, saldo, nome e cognome del proprietario (senza cointestatario), numero di movimenti associati a quel conto. Inoltre, ritornare il patrimonio complessivo della banca, cioè la somma dei saldi di tutti i conti registrati a sistema
  - IN → nessuno
  - OUT → la lista dei conti correnti o una lista vuota se non ci sono conti
  - EXCEPTION → nessuna

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Leggere saldo e ultimi movimenti di un conto corrente**
  - DESCRIZIONE → recuperare il saldo e gli ultimi movimenti di un conto. Gli ultimi movimenti sono tutti i movimenti effettuati sul conto nel mese corrente.
  - IN → numeroConto – il numero del conto da recuperare
  - OUT → il saldo più gli ultimi movimenti del conto
  - EXCEPTION → nessuna

- **Controllare la validità di un conto**
  - DESCRIZIONE → controllare che i dati presenti a sistema di un conto corrente siano validi. In particolare, controllare che:
    - La sommatoria degli importi di tutti i movimenti del conto sia coerente con il saldo del conto.
    - Tutti i movimenti siano effettuati solo ed esclusivamente dai proprietari del conto
    - Il saldo non deve essere inferiore a -5000
    - Ci sia almeno un utente in qualità di proprietario

  - IN → numeroConto – il numero del conto soggetto al controllo
  - OUT → l'esito del controllo. In particolare: l'esito (true o false). Nel caso di false, un messaggio che faccia capire il motivo del fallimento.
  - EXCEPTION → se il conto con numeroConto non esiste sollevare un errore

Descrizione generale dei servizi back-end

Sviluppare una applicazione back end con Spring e Spring Boot per offrire i seguenti servizi di business:

Servizi sul conto corrente

- **Sganciare il cointestatario di un conto**
  - DESCRIZIONE → cancellare l'associazione tra il cointestatario e il conto corrente. ATTENZIONE: non cancellare l'utente ma solo l'associazione tra quell'utente ed il conto. E' possibile sganciare il cointestatario del conto solo se non ha effettuato nessuna operazione sul conto (cioè nessun movimento).
  - IN →
    - numeroConto – il numero del conto da recuperare
    - idUtenteCointestatario – l'identificativo dell'utente cointestatario
  - OUT → i dati salienti del contoCorrente dopo la modifica.
  - EXCEPTION → se il numero del conto non esiste oppure se idUtenteCointestatario non esiste oppure se idUtenteCointestatario non è il cointestatario del conto, sollevare un errore. Inoltre, se l'utente da sganciare ha effettuato anche un solo movimento, sollevare una eccezione.

Buon lavoro